PÓS TECH

DEVOPS E ARQUITETURA CLOUD

# FASE 4 | AULA 10 -

# NEW RELIC - PROGRAMABILIDADE E "FULL-STACK" (APPLIED INTELLIGENCE, NERDLETS & RUM)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

**SUMÁRIO**

O QUE VEM POR AI? ..... 3
HANDS ON ..... 4
SAIBA MAIS ..... 5
O QUE VOCÊ VIU NESTA AULA? ..... 26
REFERÊNCIAS ..... 27

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

## O QUE VEM POR AI?

Esta aula aborda os conceitos avançados de observabilidade na plataforma New Relic, com foco na programabilidade da plataforma, na aplicação de AIOps por meio do Applied Intelligence e na construção de uma visão full-stack da experiência do sistema, integrando backend, frontend e inteligência artificial.

São apresentados os fundamentos de detecção automática de anomalias, correlação de incidentes e análise de causa raiz, além da extensibilidade da plataforma por meio de Nerdlets, Nerdpacks e da API NerdGraph (GraphQL).

A aula também introduz o Real User Monitoring (RUM) como elemento essencial para compreender a performance percebida pelos usuários finais, conectando métricas técnicas à experiência real de uso da aplicação.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

New Relic - Programabilidade e "Full-Stack" Página 4 de 29

## HANDS ON

As atividades práticas desta aula explorarão a aplicação direta dos conceitos de observabilidade programável e full-stack. Serão realizadas simulações de anomalias em aplicações .NET para análise automática de incidentes com Applied Intelligence, incluindo detecção de desvios, correlação de alertas e identificação de causa raiz.

Na sequência, serão desenvolvidas extensões personalizadas da interface do New Relic por meio da criação de Nerdpacks e Nerdlets, incluindo a associação contextual dessas aplicações a entidades monitoradas e o consumo de dados via NerdGraph.

Por fim, será habilitado o Real User Monitoring, que permitirá a análise integrada de métricas de frontend e backend, com destaque para Core Web Vitals, correlação entre RUM e APM e validação do impacto de gargalos técnicos na experiência do usuário final.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

# SAIBA MAIS

## Applied Intelligence

### Detecção de Anomalias e Análise de Causa Raiz

A plataforma New Relic utiliza Applied Intelligence para oferecer uma suíte robusta de funcionalidades de AIOps (Inteligência Artificial para Operações de TI), cujo objetivo é automatizar a detecção de problemas, acelerar a análise de causa raiz e reduzir o tempo médio de resolução (MTTR). O pilar dessa capacidade é a detecção proativa de anomalias, que monitora continuamente os dados de telemetria em busca de desvios do comportamento normal.

O serviço analisa sinais vitais de aplicações e infraestrutura, como latência, taxa de erros, throughput e utilização de CPU, aplicando modelos de Machine Learning para aprender os padrões sazonais e de linha de base de cada métrica. Quando um desvio estatisticamente significativo é identificado, a plataforma o classifica como uma anomalia e gera um alerta, muitas vezes antes que os limiares estáticos tradicionais sejam violados. Este processo é fundamental para reduzir o "tempo para detectar" (MTTD), permitindo que as equipes de operações ajam antes que o problema impacte um número significativo de usuários.

Uma vez que um incidente é criado, a análise de causa raiz entra em ação. A New Relic correlaciona automaticamente múltiplos incidentes e eventos relacionados em um único "issue" para evitar a fadiga de alertas. A plataforma gera um mapa de impacto (issue map), uma visualização topológica que mostra a entidade problemática no centro e suas dependências diretas e indiretas. Este mapa contextualiza o incidente, exibindo como a falha em um serviço pode estar afetando outros componentes do sistema, como aplicações upstream ou downstream, hosts de infraestrutura ou bancos de dados.

Além do mapa, a análise de causa raiz da New Relic examina os dados de telemetria (métricas, eventos, logs e traços) das entidades envolvidas, momentos antes e durante o incidente. A plataforma destaca os atributos e eventos que mais se correlacionam com a anomalia, como um aumento súbito em erros de uma transação específica, uma alteração de configuração recente (deploy) ou uma métrica de

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

infraestrutura saturada. Essa análise direcionada fornece uma "causa provável", guiando os engenheiros diretamente para a origem do problema e eliminando a necessidade de investigar manualmente múltiplos dashboards e logs em busca de pistas.

Segundo a documentação da New Relic, a inteligência aplicada "identifica anomalias de forma contínua, reduz o ‘tempo para detectar’ (MTTD) e fornece alertas em tempo real de falhas, oferecendo uma visualização única dos incidentes".

Em resumo, a combinação de detecção de anomalias e análise de causa raiz automatizada transforma o monitoramento reativo em uma prática proativa e inteligente, permitindo que as equipes de SRE e DevOps mantenham a estabilidade e a performance de sistemas complexos com maior eficiência.

**Redução de Ruído e Decisões de Correlação**

Em ambientes de microsserviços e sistemas distribuídos, um único ponto de falha pode desencadear uma cascata de alertas em múltiplos serviços e camadas da infraestrutura. Esse fenômeno, conhecido como ruído de alertas, leva rapidamente à fadiga de alertas, uma condição em que as equipes de operações se tornam dessensibilizadas a notificações devido ao volume excessivo, aumentando o risco de ignorar um problema crítico. A New Relic aborda esse desafio através de um sistema de correlação inteligente que agrupa incidentes relacionados em um único "issue" acionável, e o principal mecanismo para customizar essa lógica são as Decisions (Decisões).

As "Decisions" são regras lógicas que instruem a plataforma sobre como agrupar incidentes. Elas podem ser sugeridas pela própria IA da New Relic, que analisa padrões de correlação em dados históricos, ou podem ser criadas manualmente pelos usuários para refletir o conhecimento específico do domínio e da arquitetura do sistema. O objetivo é consolidar uma "tempestade de alertas" em uma única notificação contextualizada, que representa a totalidade do impacto de um problema subjacente.

O processo de criação de uma "Decision" é guiado pela interface da plataforma:

**Seleção de Dados:** o usuário seleciona um conjunto de incidentes que, em sua análise, estão relacionados, mas que a plataforma não agrupou automaticamente.

FIAP
PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

New Relic - Programmability de Full-Stack Observability, Applied Intelligence, Nerdlets & RUM

**Definição da Lógica:** a New Relic analisa os atributos (metadados) dos incidentes selecionados e sugere os campos comuns que podem ser usados para a correlação. Por exemplo, alertas de diferentes fontes (APM, Infraestrutura, Logs) podem compartilhar o mesmo appName, host.name, ou um custom.attribute como region ou service.tier.

**Teste de Impacto:** antes de ativar a regra, a plataforma permite testá-la contra dados históricos. Isso fornece uma prévia de quantos incidentes teriam sido correlacionados no passado, permitindo que o usuário ajuste a precisão da regra para evitar correlações excessivas (agrupar o que não deveria) ou insuficientes.

**Ativação:** uma vez validada, a "Decision" é ativada e passa a agrupar futuros incidentes que correspondam à lógica definida.

A documentação da New Relic enfatiza as boas práticas na criação de "Decisions", recomendando "começar com poucos critérios, monitorar a taxa de correlação e ativar a detecção de flapping (alertas intermitentes) para evitar ocultar problemas". Uma regra muito ampla pode mascarar problemas distintos, enquanto uma muito restrita pode não ser eficaz na redução de ruído.

Essa capacidade de personalizar a lógica de correlação é vital. Ela permite que as equipes incorporem seu conhecimento sobre o sistema diretamente na ferramenta de monitoramento, garantindo que os "issues" gerados pela plataforma representem problemas de negócio reais e acionáveis, em vez de apenas uma coleção de sintomas técnicos desconexos. O resultado é uma redução drástica no ruído, permitindo que os engenheiros se concentrem na resolução de problemas em vez de na triagem de alertas.

**Ai Monitoring, Llms e Benefícios de Aiops**

A adoção de AIOps (Inteligência Artificial para Operações de TI) transcende a simples automação de alertas, representando uma mudança fundamental na forma como as organizações gerenciam a complexidade dos sistemas modernos. Os benefícios se estendem por todo o ciclo de vida da observabilidade, desde a detecção proativa até a otimização de custos. A New Relic define AIOps como a aplicação de IA para operar e otimizar sistemas, destacando benefícios como a melhoria da performance, redução de downtime, aceleração da análise de causa raiz e a capacidade de prever problemas futuros.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

New Relic - Programabilidade e "Full-Stack" Applied Intelligence, Nerdlets & RUM

Com a ascensão da inteligência artificial generativa, uma nova fronteira da observabilidade emergiu: o monitoramento de Modelos de Linguagem Grandes (LLMs). Aplicações que integram LLMs, como chatbots e assistentes virtuais, introduzem novas camadas de complexidade e incerteza. O desempenho dessas aplicações não depende apenas do código, mas também da qualidade, latência e custo das interações com as APIs dos LLMs (como as da OpenAI, Bedrock ou DeepSeek).

O AI Monitoring da New Relic é uma capacidade especializada do APM (Application Performance Monitoring) projetada para fornecer visibilidade completa sobre o stack de IA. Uma vez que o agente APM está instrumentado em uma aplicação que utiliza um LLM, ele pode ser configurado para capturar dados detalhados sobre cada interação com o modelo. Isso inclui:

**Prompts e Respostas:** o conteúdo exato dos prompts enviados pelos usuários e as respostas geradas pelo LLM são capturados, permitindo a análise de interações problemáticas ou ineficazes.

**Métricas de Tokens:** o número de tokens de entrada (prompt) e de saída (completion) é monitorado. Como o custo da maioria dos serviços de LLM é baseado no uso de tokens, essa métrica é crucial para o gerenciamento de custos.

**Latência e Erros:** o tempo de resposta de cada chamada ao LLM é medido, assim como a ocorrência de erros, permitindo a identificação de gargalos de performance ou falhas na integração.

**Feedback do Usuário:** a plataforma pode ser instrumentada para capturar o feedback do usuário final (por exemplo, um polegar para cima ou para baixo em uma resposta), correlacionando a satisfação do usuário com interações específicas do LLM.

A New Relic destaca que sua solução de AI Monitoring permite "ver além do LLM", rastreando fluxos de trabalho que podem envolver bancos de dados, APIs externas e outros serviços, fornecendo um contexto completo para cada transação de IA.

Esses dados são agregados em dashboards pré-configurados que oferecem uma visão geral da saúde, custo e qualidade do serviço de IA. As equipes podem comparar o desempenho de diferentes modelos, identificar os prompts mais caros, e

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

criar alertas para picos de latência ou taxas de erro. Essa visibilidade é indispensável para otimizar a experiência do usuário, controlar os custos operacionais e garantir a confiabilidade de aplicações baseadas em IA.

## Nerdlets e Programabilidade

### O Que São Nerdlets e Nerdpacks

A plataforma New Relic One foi projetada para ser totalmente programável, permitindo que as equipes de desenvolvimento estendam a interface do usuário (UI) com aplicações e visualizações personalizadas. Os blocos de construção fundamentais dessa programabilidade são os Nerdlets e os Nerdpacks. Essa arquitetura aberta transforma a plataforma de uma ferramenta de monitoramento em um ambiente de desenvolvimento para a criação de soluções de observabilidade sob medida.

Um Nerdpack é o contêiner principal de uma aplicação customizada. Ele funciona como um pacote de software que agrupa todos os arquivos necessários para a aplicação, incluindo código, metadados, dependências e recursos. A estrutura de um Nerdpack é gerenciada através da New Relic CLI (nr1), a ferramenta de linha de comando que permite criar, testar, servir e implantar aplicações na plataforma. Ao criar um Nerdpack com o comando nr1 create --type nerdpack, a CLI gera uma estrutura de diretórios padronizada, que inclui:

- nr1.json: o arquivo de manifesto do Nerdpack. Ele contém metadados essenciais como o identificador único (id), o nome de exibição (displayName) e uma descrição da aplicação.
- package.json: o arquivo padrão do Node.js para gerenciamento de dependências do projeto.
- nerdlets/: um diretório que contém o código-fonte de um ou mais Nerdlets.
- launchers/: um diretório opcional que contém as configurações para os "launchers" (iniciadores) da aplicação.

Um Nerdlet é o componente de UI em si. Trata-se de uma aplicação React que renderiza visualizações, interage com os dados da New Relic e implementa a lógica

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

de negócio da aplicação customizada. Cada Nerdlet reside em seu próprio subdiretório dentro da pasta nerdlets/ e possui sua própria estrutura de arquivos:

- index.js: o ponto de entrada do componente React, onde a lógica da UI é implementada.
- nr1.json: o arquivo de configuração específico do Nerdlet, que define seu id, displayName e, crucialmente, seu contexto de entidade (ver subtópico 2.2).
- styles.scss: um arquivo para estilização do componente usando SCSS.

Finalmente, um Launcher (iniciador) é o que torna um Nerdlet acessível diretamente a partir da página inicial de "Apps" na New Relic. Ele funciona como um atalho, associando um ícone e um nome a um Nerdlet específico. A configuração do launcher, definida em seu próprio arquivo nr1.json dentro da pasta launchers/, especifica o rootNerdletId, que aponta para o ID do Nerdlet que deve ser aberto quando o usuário clica no ícone do launcher.

A documentação oficial descreve a relação de forma clara: "Uma Nerdpack é o pacote que contém todos os arquivos de uma aplicação ou visualização... Dentro de uma Nerdpack existem Launchers (que adicionam um ícone e iniciam o app) e Nerdlets (componentes React)".

Essa estrutura modular permite a criação de aplicações complexas, onde múltiplos Nerdlets podem ser combinados para criar diferentes telas ou funcionalidades, todos empacotados e gerenciados como uma única Nerdpack. A CLI nr1 abstrai a complexidade do build e do deploy, permitindo que os desenvolvedores se concentrem na criação de valor com React e na consulta de dados de observabilidade.

**Atribuir Nerdlets a Entidades**

Uma das características mais poderosas da programabilidade da New Relic é a capacidade de injetar aplicações customizadas (Nerdlets) diretamente no contexto de entidades monitoradas. Em vez de existirem como aplicações isoladas, os Nerdlets podem ser anexados a entidades, aparecendo como uma nova aba ou opção de visualização dentro do Entity Explorer. Isso permite a criação de dashboards, ferramentas de diagnóstico ou visualizações de dados que são contextualmente

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

New Relic - Programabilidade "Full-Stack" Applied Intelligence, Nerdlets & RUM

relevantes para a entidade específica que está sendo analisada, seja uma aplicação, um host, um container ou um serviço de nuvem.

O mecanismo para essa associação é a configuração do arquivo nr1.json do próprio Nerdlet. Ao adicionar uma seção context a este arquivo, a pessoa desenvolvedora pode especificar a quais tipos de entidades o Nerdlet deve ser associado. A configuração context.entities aceita uma lista de objetos, cada um definindo um domain (domínio) e um type (tipo) de entidade.

Os domínios correspondem às principais categorias de monitoramento da New Relic:

- APM: aplicações monitoradas pelo agente APM.
- BROWSER: aplicações monitoradas pelo agente de Browser (RUM).
- INFRA: entidades de infraestrutura, como hosts e containers.
- MOBILE: aplicações mobile.
- SYNTH: monitores sintéticos.

Dentro de cada domínio, existem tipos específicos. Por exemplo, no domínio APM, o tipo pode ser APPLICATION. No domínio INFRA, os tipos podem ser HOST ou CONTAINER. É possível até mesmo usar a negação (prefixo!) para excluir domínios, tornando o Nerdlet visível em todas as entidades, exceto nas especificadas.

**JSON**

```
    { "schemaType": "NERDLET", "id": "my-contextual-nerdlet",
"displayName":     "Custom APM View", "description": "A custom
view for APM applications",       "context": { "entities": [ {
"domain": "APM", "type": "APPLICATION" } ] },
    "actionCategory": "monitor" }
```

Código-fonte 1 – Json
Fonte: Elaborado pelo autor (2026)

No exemplo acima, o Nerdlet my-contextual-nerdlet aparecerá como uma opção de navegação para qualquer entidade do tipo APPLICATION dentro do domínio APM.

Além do context, a propriedade actionCategory define onde na interface da entidade o link para o Nerdlet será exibido. Categorias comuns incluem monitor

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

New Relic - Programabilidade e "Full-Stack" Application Intelligence, Nerdlets & RUM

(geralmente na navegação lateral principal) ou overview (na página de resumo). Isso permite um controle granular sobre a integração da UI, tornando a experiência do usuário mais fluida e intuitiva.

Segundo a documentação, "múltiplos domínios e tipos podem ser listados; isso controla em quais tipos de entidades o Nerdlet aparecerá e também habilita ações contextuais". Quando um Nerdlet é aberto a partir do contexto de uma entidade, ele recebe automaticamente informações sobre essa entidade (como seu guid e accountId), permitindo que a aplicação customizada execute consultas e exiba dados que são relevantes apenas para aquela entidade específica, sem a necessidade de o usuário selecioná-la manualmente.

Essa capacidade de anexar Nerdlets a entidades é o que permite a criação de verdadeiras "lentes" de observabilidade, fornecendo às equipes as ferramentas exatas de que precisam, precisamente onde precisam delas, para analisar e resolver problemas de forma mais eficaz.

**Consultar Dados com Nerdgraph (Graphql)**

O coração de qualquer aplicação de observabilidade é o acesso aos dados de telemetria. Em aplicações customizadas da New Relic, a interação com os dados da plataforma é realizada através do NerdGraph, a API baseada em GraphQL da New Relic. O GraphQL oferece uma alternativa flexível e poderosa às APIs REST tradicionais, permitindo que os clientes (neste caso, os Nerdlets) solicitem precisamente os dados de que necessitam, nem mais, nem menos.

As principais vantagens do NerdGraph sobre as APIs REST convencionais são:

**Eficiência de Dados:** o cliente especifica exatamente quais campos e objetos ele deseja receber na resposta. Isso evita o over-fetching (receber dados desnecessários) e o under-fetching (ter que fazer múltiplas chamadas para obter todos os dados necessários), resultando em uma comunicação de rede mais eficiente.

**Agregação de Fontes:** uma única consulta NerdGraph pode buscar dados de múltiplas fontes dentro da New Relic. É possível, em uma só chamada, obter informações da conta do usuário, detalhes de uma entidade monitorada e os resultados de uma consulta NRQL (New Relic Query Language).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

**API Fortemente Tipada:** o esquema do NerdGraph define rigorosamente os tipos de dados, campos e operações disponíveis. Isso permite a introspecção da API e habilita ferramentas de desenvolvimento poderosas, como o autocompletar e a validação de consultas em tempo de desenvolvimento.

Para facilitar a integração de consultas NerdGraph em Nerdlets, o SDK da New Relic fornece o componente React NerdGraphQuery. Este componente abstrai a complexidade de configurar um cliente Apollo (a biblioteca GraphQL subjacente) e gerenciar o estado da solicitação.

O NerdGraphQuery utiliza o padrão de render prop, onde seu filho é uma função que recebe o estado atual da consulta e retorna o JSX a ser renderizado. Essa função recebe um objeto com três propriedades principais:

- loading: um booleano que é true enquanto a consulta está em andamento.
- error: um objeto que contém informações de erro, caso a consulta falhe.
- data: o objeto com os dados da resposta, uma vez que a consulta seja concluída com sucesso.

**JavaScript**

```
    import React from 'react';
    import { NerdGraphQuery, Spinner } from 'nr1';
    const MY_QUERY = ` query($accountId: Int!) { actor { account(id: $accountId) { name nrql(query: "SELECT average(duration) FROM Transaction SINCE 1 hour ago") { results } } } } `;
    function MyNerdlet({ accountId }) { return ( <NerdGraphQuery query= {MY_QUERY} variables={{ accountId }}> {({ loading, error, data }) => { if (loading) { return <Spinner />; } if (error) { return <p>Error loading data.</p>; } const accountName = data.actor.account.name; const avgDuration =
    data.actor.account.nrql.results[0].average; return ( <div> <h2>{accountName} </h2> <p>Average Transaction Duration: {avgDuration.toFixed(3)}s</p> </div> ); }} </NerdGraphQuery> ); }
```

Código-fonte 2 – JavaScript
Fonte: Elaborado pelo autor (2026)

Como demonstrado no exemplo acima e na documentação, o componente NerdGraphQuery permite "incorporar consultas GraphQL dentro de um Nerdlet". A

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

consulta (query) e as variáveis (variables) são passadas como props, e o componente gerencia o ciclo de vida da requisição, simplificando drasticamente o desenvolvimento de aplicações ricas em dados.

Essa abordagem permite que os(as) desenvolvedores(as) construam dashboards e ferramentas altamente personalizados, combinando dados de performance de aplicações, infraestrutura, experiência do usuário e logs em uma única interface coesa, tudo dentro da plataforma New Relic.

## Rum (Real User Monitoring)

### Rum vs. Apm

No universo da observabilidade, Application Performance Monitoring (APM) e Real User Monitoring (RUM) são duas disciplinas complementares, mas distintas, que fornecem visões de ponta a ponta do desempenho de uma aplicação. Enquanto o APM se concentra na saúde e na performance do backend (lado do servidor), o RUM foca na experiência real do usuário final no frontend (lado do cliente). Compreender a diferença e a sinergia entre ambos é fundamental para uma estratégia de monitoramento full-stack.

O APM instrumenta o código da aplicação no servidor, em linguagens como .NET, Java, Python ou Node.js. Seus agentes coletam métricas detalhadas sobre a execução do código, como:

- **Latência de Transações:** o tempo que o servidor leva para processar uma requisição web ou uma tarefa em background.
- **Taxa de Erros:** a frequência com que ocorrem exceções não tratadas ou erros HTTP no servidor.
- **Throughput**: o volume de requisições que a aplicação está processando por minuto.
- **Rastreamento Distribuído (Distributed Tracing):** a capacidade de seguir uma única requisição através de múltiplos microsserviços para identificar gargalos em sistemas complexos.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Métricas de Recursos:** uso de CPU, memória e outras métricas do ambiente de execução.

**Essencialmente, o APM responde à pergunta: "Quão rápido e confiável é o meu código no servidor?"**

Por outro lado, o RUM foca na experiência real do usuário, capturando dados diretamente do navegador do cliente. Um snippet de JavaScript, injetado nas páginas da aplicação, utiliza as APIs de performance do navegador para coletar métricas que refletem o que o usuário de fato percebe, como:

**Tempos de Carregamento de Página**: o tempo total para a página se tornar interativa, incluindo a resolução de DNS, a conexão de rede, o tempo de resposta do servidor (time to first byte) e a renderização no navegador.

**Core Web Vitals**: métricas do Google (LCP, INP, CLS) que quantificam a experiência de carregamento, interatividade e estabilidade visual.

**Erros de JavaScript:** exceções que ocorrem no navegador e que podem quebrar a funcionalidade da página.

**Performance de Requisições AJAX:** a latência de chamadas assíncronas que buscam dados após o carregamento inicial da página.

**Dados de Sessão:** informações sobre o navegador, dispositivo, sistema operacional e localização geográfica do usuário.

**O RUM responde à pergunta: "Quão rápida e funcional é a minha aplicação para os usuários em seus diferentes contextos?"**

A New Relic integra os dados de RUM e APM, permitindo que as equipes correlacionem problemas de frontend com suas causas no backend. Por exemplo, um tempo de carregamento de página lento (detectado pelo RUM) pode ser rastreado até uma transação de servidor lenta ou uma consulta de banco de dados ineficiente (identificada pelo APM), fornecendo uma visibilidade completa do ciclo de vida da requisição.

A tabela a seguir resume as principais diferenças entre as duas abordagens:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

| Área | RUM (Real User Monitoring) | APM (Application Performance Monitoring) |
|---|---|---|
| **Foco** | Experiência do usuário final (client-side) | Desempenho da aplicação (server-side) |
| **Origem dos Dados** | Navegador do usuário (via JavaScript) | Agentes no servidor da aplicação |
| **Métricas Chave** | Core Web Vitals, tempos de carregamento, erros de JS | Latência de transação, taxa de erros, throughput |
| **Visibilidade** | Desempenho percebido pelo usuário, segmentado por dispositivo, navegador, etc. | Gargalos de código, consultas lentas a banco de dados, problemas de infraestrutura |
| **Instrumentação** | Snippet de JavaScript nas páginas HTML | Instalação de agente no ambiente do servidor |

Figura 1 – Diferenças entre RUM e APM
Fonte: Elaborado pelo autor (2026)

### Como Funciona o Rum e Quais Métricas São Coletadas

O Real User Monitoring (RUM) da New Relic opera através da instrumentação do frontend da aplicação com um agente JavaScript. Esse agente é um pequeno snippet de código que é inserido no cabeçalho (<head>) das páginas HTML. Uma vez carregado no navegador do usuário, o agente utiliza APIs de performance nativas do navegador, como a Navigation Timing API, a Resource Timing API e a Performance API, para coletar dados detalhados sobre a experiência de carregamento e interação em tempo real.

O processo de coleta de dados pode ser resumido nos seguintes passos:

**Instrumentação**: o snippet de JavaScript é adicionado às páginas da aplicação. Isso pode ser feito de duas maneiras principais:

- **Auto-injeção pelo Agente APM**: se a aplicação backend já estiver monitorada por um agente APM da New Relic (como o agente .NET), ele pode ser configurado para injetar automaticamente o script do RUM em todas as páginas renderizadas. Esta é a abordagem mais comum e recomendada.
- **Instrumentação Manual:** o snippet pode ser copiado e colado manualmente no template HTML da aplicação.

**Coleta de Métricas**: quando um usuário visita a página, o agente RUM começa a coletar uma vasta gama de métricas, que incluem:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Page Load Timing:** o ciclo de vida completo do carregamento da página, desde o início da navegação até o evento window.load. Isso é decomposto em fases como tempo de rede, tempo de backend (Time to First Byte) e tempo de frontend (renderização do DOM).
- **Resource Timing:** o tempo de carregamento de recursos individuais da página, como imagens, scripts CSS e fontes.
- **AJAX Requests:** a latência e o status de requisições assíncronas (AJAX/Fetch) feitas após o carregamento inicial da página, cruciais para Single Page Applications (SPAs).
- **JavaScript Errors:** captura de exceções de JavaScript não tratadas que ocorrem no navegador, incluindo o stack trace completo para depuração.
- **Core Web Vitals:** métricas de experiência do usuário definidas pelo Google (LCP, INP, CLS).
- **Session Traces:** uma linha do tempo detalhada do carregamento de uma sessão de usuário individual, mostrando todos os eventos e recursos em cascata.

**Envio de Dados:** os dados coletados são agrupados e enviados de forma assíncrona para a plataforma New Relic em intervalos regulares. Este envio é otimizado para ter um impacto mínimo na performance da aplicação cliente.

**Processamento e Visualização:** na plataforma, os dados são processados, agregados e disponibilizados em dashboards interativos, permitindo a análise e a criação de alertas.

A importância do RUM reside em sua capacidade de medir o desempenho real, conforme percebido pelos usuários. Como a New Relic ressalta, "a experiência do usuário está correlacionada com a retenção; cerca de 40% dos usuários abandonam um site se uma página demora mais de 3 segundos para carregar". O RUM permite que as equipes identifiquem e priorizem exatamente esses gargalos que afetam diretamente a satisfação do cliente e os resultados de negócio.

Ao capturar dados de uma população diversificada de usuários, com diferentes dispositivos, navegadores, sistemas operacionais e condições de rede, o RUM

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

oferece uma visão muito mais realista do desempenho do que testes sintéticos realizados em ambientes controlados.

**Core Web Vitals, Dashboards e Otimização**

Para quantificar a experiência do usuário de forma padronizada e acionável, o Google introduziu a iniciativa Core Web Vitals. Estas são um conjunto de métricas focadas em três aspectos da experiência do usuário: carregamento, interatividade e estabilidade visual. A New Relic integra nativamente a coleta e a visualização dessas métricas em seu produto de Browser Monitoring (RUM), tornando-as um pilar para a otimização do desempenho do frontend.

As três Core Web Vitals são:

**Largest Contentful Paint (LCP):** mede o desempenho do carregamento. Ela reporta o tempo de renderização do maior elemento de conteúdo (imagem ou bloco de texto) visível na janela de visualização. Um bom LCP (abaixo de 2.5 segundos) indica que o conteúdo principal da página é carregado rapidamente, dando ao usuário a percepção de que a página é útil.

**Interaction to Next Paint (INP):** mede a interatividade ou a capacidade de resposta da página. O INP observa a latência de todas as interações de clique, toque e teclado que um usuário faz com a página e reporta a pior latência observada (ou quase a pior). Um INP baixo (abaixo de 200 milissegundos) significa que a página responde rapidamente às entradas do usuário, proporcionando uma experiência fluida.

**Cumulative Layout Shift (CLS):** mede a estabilidade visual. Ela quantifica a quantidade de mudanças inesperadas de layout que ocorrem durante o ciclo de vida da página. Um CLS baixo (abaixo de 0.1) garante que os elementos da página não se movam inesperadamente, evitando que os usuários cliquem em algo por engano.

O painel Summary do Browser Monitoring na New Relic exibe essas métricas de forma proeminente, mostrando não apenas os valores médios, mas também a distribuição de percentis (como p75 e p95), que são mais representativos da experiência da maioria dos usuários do que uma simples média. A plataforma também mostra as tendências dessas métricas ao longo do tempo, permitindo que as equipes vejam o impacto de otimizações ou a introdução de regressões após um deploy.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

A documentação da New Relic destaca que a plataforma "permite segmentar dados por navegador, dispositivo, sistema operacional ou localização geográfica". Essa capacidade de filtragem é crucial para a otimização.

Por exemplo, uma equipe pode descobrir que o LCP é alto apenas para usuários em dispositivos móveis em uma determinada região. Com essa informação, eles podem investigar se o problema está relacionado ao tamanho das imagens servidas para telas menores ou à latência da rede naquela localidade. Da mesma forma, um INP alto pode ser correlacionado a um navegador específico, apontando para um problema de compatibilidade no código JavaScript.

O processo de otimização geralmente segue este fluxo:

**Identificação:** usar os dashboards da New Relic para identificar qual métrica (LCP, INP ou CLS) está abaixo da meta e para qual segmento de usuários (dispositivo, navegador, página específica).

**Diagnóstico:** aprofundar a investigação. Se o LCP é alto, a New Relic pode ajudar a determinar se o gargalo está no backend (tempo de resposta do servidor) ou no frontend (tempo de renderização). Se o INP é alto, os stack traces de erros de JavaScript ou a análise de transações de SPAs podem revelar a causa.

**Ação:** implementar as correções necessárias, como otimizar imagens, reduzir o bloqueio da thread principal por JavaScript ou pré-carregar fontes.

**Validação:** monitorar os dashboards após a implementação para confirmar que a otimização teve o efeito desejado nas Core Web Vitals.

Além dos dashboards pré-configurados, os dados das Core Web Vitals estão disponíveis para consulta via NRQL, permitindo a criação de painéis totalmente personalizados e a configuração de alertas para quando essas métricas críticas de experiência do usuário se degradarem.

## Synthetic Monitoring

### Boas Práticas de Synthetic Monitoring

O Synthetic Monitoring (Monitoramento Sintético) é uma abordagem proativa para a observabilidade, que consiste em simular o comportamento do usuário e as

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

chamadas de API por meio de scripts automatizados. Diferente do RUM, que mede a experiência de usuários reais, o monitoramento sintético executa testes em intervalos regulares a partir de locais controlados ao redor do mundo, permitindo detectar problemas antes que eles afetem os clientes. Para maximizar a eficácia dessa abordagem, a New Relic recomenda um conjunto de boas práticas.

**Escolha do Monitor Apropriado**: a eficácia do monitoramento sintético começa com a seleção do tipo de monitor correto para o objetivo do teste. A New Relic oferece vários tipos, cada um com um propósito específico:

- Ping: o mais simples. Verifica a disponibilidade de um endpoint, confirmando que ele está online e respondendo a requisições. Ideal para verificar a saúde básica de servidores e serviços críticos.
- Simple Browser: carrega uma página web completa em uma instância real do navegador Chrome, medindo o tempo de carregamento e a disponibilidade. É útil para monitorar o desempenho de páginas importantes, como a home page ou páginas de destino.
- Step Monitor (Codeless): permite criar fluxos de usuário de múltiplos passos através de uma interface visual, sem a necessidade de escrever código. É ideal para validar jornadas críticas, como processos de login, pesquisa de produtos ou adição de itens ao carrinho.
- API Test: envia requisições HTTP para endpoints de API, permitindo validar não apenas a disponibilidade, mas também a correção das respostas, verificando status codes, headers e o conteúdo do corpo da resposta. Essencial para monitorar a saúde de microsserviços e APIs de backend.
- Scripted Browser: o tipo mais poderoso. Utiliza scripts baseados em Selenium WebDriver para emular fluxos de usuário complexos e interações dinâmicas que não podem ser cobertas por monitores de passo.

**Configuração de Múltiplas Localizações**: executar monitores a partir de uma única localização pode levar a falsos positivos causados por problemas de rede localizados. A recomendação é configurar cada monitor para ser executado a partir de, no mínimo, três localizações geográficas distintas. Se um teste falhar em apenas

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

uma localização, é provável que seja um problema de rede regional. Se falhar em todas, é um forte indicativo de um problema real na aplicação.

**Uso de Private Locations**: para aplicações e endpoints que não são expostos à internet pública (como ambientes de staging ou serviços internos atrás de um firewall), a New Relic oferece as Private Locations. Isso envolve a instalação de um software (um minion em um container Docker) na rede privada do cliente, que executa os mesmos testes sintéticos, mas a partir de dentro do perímetro de segurança da empresa.

**Ajuste da Frequência de Execução**: a frequência com que um monitor é executado deve ser um balanço entre a necessidade de detecção rápida e o custo (já que cada execução tem um custo associado). Serviços críticos podem exigir verificações a cada minuto, enquanto páginas menos importantes podem ser testadas a cada 15 ou 30 minutos.

O guia de boas práticas da New Relic aconselha a "selecionar o tipo de monitor mais adequado... definir várias localidades para reduzir falsos positivos, configurar verificação de SSL... e ajustar a frequência de execução".

Seguir essas práticas garante que o monitoramento sintético forneça sinais confiáveis e acionáveis, atuando como um sistema de alerta precoce que aumenta a confiabilidade e a resiliência da aplicação.

**Usando Monitores e Tipos De Monitor**

A plataforma de Synthetic Monitoring da New Relic oferece um arsenal de tipos de monitores para simular diferentes tipos de interações e verificar a saúde de uma aplicação de ponta a ponta. Cada tipo de monitor é projetado para um caso de uso específico, desde simples verificações de disponibilidade até a validação de fluxos de negócio complexos.

**Ping Monitors**: este é o tipo de monitor mais fundamental. Ele envia uma requisição HTTP HEAD ou GET para um URL especificado e mede o tempo de resposta. Seu único propósito é verificar se um endpoint está online e acessível. Monitores de ping são leves, baratos e ideais para garantir a disponibilidade contínua de serviços críticos, APIs ou websites. Eles podem ser configurados para validar o código de status da resposta e, opcionalmente, procurar por um texto específico no corpo da resposta para uma verificação de saúde mais robusta.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

**Simple Browser Monitors**: este monitor vai um passo além do ping. Ele utiliza uma instância real do navegador Google Chrome para carregar uma página web completa, incluindo todos os seus recursos (imagens, CSS, JavaScript). O resultado é uma medição realista do tempo de carregamento da página (equivalente ao evento window.onload). É a maneira mais simples de monitorar o desempenho de uma página web do ponto de vista de um usuário que a visita pela primeira vez. Ele captura uma captura de tela da página carregada e uma análise detalhada do tempo de carregamento em cascata (waterfall).

**Step Monitors (Codeless):** para validar jornadas de usuário que envolvem múltiplos passos, mas sem a complexidade de escrever código, a New Relic oferece os Step Monitors. Através de uma interface visual, o usuário pode gravar uma sequência de ações, como:

- Navegar para um URL;
- Clicar em um elemento (identificado por seu texto, seletor CSS ou XPath);
- Digitar texto em um campo de formulário;
- Validar que a navegação levou ao URL correto ou que um texto específico está presente na página.

Esses monitores são perfeitos para testar fluxos de negócio essenciais, como o processo de login, a funcionalidade de pesquisa ou a adição de um produto ao carrinho de compras, garantindo que essas jornadas críticas estejam sempre funcionando.

**API Tests**: focados no backend, os monitores de API permitem enviar requisições HTTP para endpoints de API e validar suas respostas. Eles suportam todos os principais métodos HTTP (GET, POST, PUT, DELETE, etc.), a configuração de headers customizados e a inclusão de um corpo de requisição. A validação pode ir além do código de status, permitindo asserções complexas sobre o corpo da resposta usando JSONPath ou expressões regulares. Isso é crucial para garantir a integridade e o contrato de APIs internas e externas.

A documentação descreve os monitores como "'crash test dummies' que simulam usuários". Eles atuam como sentinelas, patrulhando continuamente a aplicação em busca de problemas, independentemente do tráfego de usuários reais.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Para ambientes internos, todos esses monitores podem ser executados a partir de Private Locations, garantindo a mesma capacidade de teste para sistemas que não estão expostos à internet.

A combinação estratégica desses diferentes tipos de monitores permite que uma organização construa uma cobertura de testes sintéticos abrangente, validando desde a disponibilidade básica da infraestrutura até a funcionalidade completa da experiência do usuário e a integridade das APIs de backend.

**Monitores de Navegador com Script**

Para os cenários de teste mais complexos e dinâmicos, que exigem lógica condicional, manipulação de dados ou interações que não podem ser capturadas por monitores de passo (codeless), a New Relic oferece os Scripted Browser Monitors. Este é o tipo de monitor sintético mais flexível e poderoso, pois permite a execução de scripts customizados que emulam jornadas de usuário sofisticadas em um ambiente de navegador real e controlado.

Os scripted browser monitors são baseados no Selenium WebDriver, o padrão de fato da indústria para automação de navegadores. Os scripts são escritos em JavaScript e utilizam uma versão customizada da API do WebDriver.js, fornecendo um controle granular sobre o navegador. Isso permite que os desenvolvedores e engenheiros de SRE criem testes que podem:

- **Navegar por fluxos complexos**: realizar sequências de ações que dependem do estado da aplicação.
- **Interagir com elementos dinâmicos**: lidar com elementos que aparecem na página de forma assíncrona, como modais, pop-ups ou conteúdo carregado via AJAX.
- **Realizar asserções complexas**: validar não apenas a presença de um elemento, mas seu conteúdo, atributos ou estado (por exemplo, se um botão está desabilitado).
- **Utilizar lógica de programação:** implementar laços, condicionais, e manipular dados para simular cenários de teste variados.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Gerenciar credenciais seguras**: fazer login em aplicações utilizando credenciais armazenadas de forma segura na New Relic, sem expô-las no código do script.

O ambiente de script da New Relic fornece objetos e variáveis globais para facilitar a interação com o navegador e a plataforma:

**$browser ou $webDriver:** o objeto principal do WebDriver, que fornece métodos para controlar o navegador, como get() (para navegar para um URL), findElement() e findElements().

**$driver ou $selenium:** o objeto que expõe utilitários do Selenium, como By (para selecionar elementos por ID, nome, classe, seletor CSS ou XPath) e until (para criar condições de espera explícitas).

**$secure:** um objeto para acessar credenciais seguras que foram previamente cadastradas na plataforma, como $secure.MY_USERNAME.

Um conceito fundamental na escrita de scripts é o uso de esperas explícitas (explicit waits). Como as aplicações web modernas são altamente assíncronas, um script não pode simplesmente tentar interagir com um elemento imediatamente após uma ação. Ele deve esperar até que o elemento esteja presente e visível. A palavra-chave await em conjunto com as condições de $selenium.until é usada para garantir que o script espere de forma robusta pela condição desejada antes de prosseguir, evitando falhas intermitentes.

**JavaScript**

```
    // Importar módulos necessários const { By, until } =
require('selenium-      webdriver');
    //   Navegar   para   a   página   de   login   await
$browser.get('https://minha-          app.com/login' );
    // Encontrar o campo de usuário, esperar até que esteja
visível e digitar o          nome de usuário const userInput =
await
    $browser.wait(until.elementLocated(By.id('username')),
10000); await      userInput.sendKeys($secure.APP_USERNAME);
    // Fazer o mesmo para o campo de senha const passInput =
await              $browser.findElement(By.id('password'));
await
    passInput.sendKeys($secure.APP_PASSWORD);
    // Clicar no botão de login  await
$browser.findElement(By.id('login-          button')).click();
```

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

```
    // Esperar pela página de dashboard e validar que o título está correto await $browser.wait(until.titleIs('Meu Dashboard'), 15000);
```

Código-Fonte 3 - JavaScript
Fonte: Elaborado pelo autor (2026)

A documentação descreve que para criar um monitor, "basta abrir o recurso Synthetic Monitoring, escolher User flow/functionality, selecionar localizações, configurar tempo de timeout e escrever o script". Após cada execução, o monitor captura não apenas o resultado de sucesso ou falha, mas também logs detalhados do console, uma análise em cascata (waterfall) do carregamento dos recursos e capturas de tela em cada passo, fornecendo um contexto de depuração extremamente rico em caso de falha.

Os scripted browser monitors são a ferramenta definitiva para garantir que as jornadas de usuário mais críticas e complexas estejam funcionando perfeitamente, 24 horas por dia, 7 dias por semana.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

## O QUE VOCÊ VIU NESTA AULA?

Esta aula explorou os fundamentos de **Applied Intelligence e AIOps**, destacando como a New Relic utiliza detecção automática de anomalias e correlação inteligente de incidentes para reduzir o ruído operacional. O conteúdo também avançou para o **AI Monitoring**, focado especificamente na observabilidade de aplicações baseadas em LLMs, garantindo que modelos de inteligência artificial operem com performance e confiabilidade.

No campo da personalização, abordamos a **programabilidade da plataforma** através de Nerdpacks e Nerdlets, utilizando a CLI nr1 para gerenciar o ciclo de desenvolvimento e manifestos. Aprendemos a associar esses componentes ao contexto das entidades e a consumir dados de observabilidade de forma flexível via **NerdGraph (GraphQL)**, permitindo a criação de visões totalmente customizadas para o negócio.

Por fim, discutimos as diferenças complementares entre **APM e RUM**, detalhando o funcionamento do Real User Monitoring para a coleta de métricas de frontend. O foco foi entender como a análise de **Core Web Vitals** e a correlação entre a experiência real do usuário e o desempenho do backend permitem identificar gargalos que impactam diretamente a satisfação do cliente final.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

# REFERÊNCIAS

NEWRLIC. **AI Monitoring.** Disponível em: https://docs.newrelic.com/docs/ai-monitoring/intro-to-ai-monitoring/. Acesso em: 28 jan. 2026.

NEWRLIC. **Applied Intelligence.** Disponível em: https://newrelic.com/platform/applied-intelligence. Acesso em: 28 jan. 2026.

NEWRLIC. **Browser Monitoring – Introduction.** Disponível em: https://docs.newrelic.com/docs/browser/browser-monitoring/getting-started/introduction-browser-monitoring/. Acesso em: 28 jan. 2026.

NEWRLIC. **Build a Nerdpack.** Disponível em: https://docs.newrelic.com/docs/new-relic-solutions/build-nr-ui/build-ab-app/create-nerdpack/. Acesso em: 28 jan. 2026.

NEWRLIC. **Core Web Vitals.** Disponível em: https://newrelic.com/platform/browser-monitoring. Acesso em: 28 jan. 2026.

NEWRLIC. **NerdGraph API Introduction.** Disponível em: https://docs.newrelic.com/docs/apis/nerdgraph/get-started/introduction-new-relic-nerdgraph/. Acesso em: 28 jan. 2026.

NEWRLIC. **Nerdpack file structure.** Disponível em: https://docs.newrelic.com/docs/new-relic-solutions/build-nr-ui/nerdpack-file-structure/. Acesso em: 28 jan. 2026.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

**PALAVRAS-CHAVE**

Observabilidade Full-Stack, Programabilidade, AIOps

FIAP

**PDF exclusivo para Gabriel Pinelli Silva - rm373763**
**gabrieltocaccelli@gmail.com**




PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com